

Phase 3 — Graduation Project

AI-Powered Test Case Generator — Full Pipeline

Goal

Build a complete end-to-end pipeline that reads a user story from a `.txt` file, fills a prompt template at runtime, calls Gemini API with retry logic, validates the JSON response defensively, prints a formatted report, saves to `.json`, and POSTs each test case to JSONPlaceholder — combining every concept from Phase 1 and Phase 2 into one working graduation script.

Step by Step — What Your Script Must Do

STEP 1 — Load API key safely Phase 2 Week 2

- Use `python-dotenv` to load `GEMINI_API_KEY` from `.env`
- If key is missing or empty → raise `MissingAPIKey` custom exception → exit cleanly
- Print `[STEP 1] API key loaded ... Done ✓`

STEP 2 — Read user story from .txt file Phase 1 Week 3

- Read `user_story.txt` from disk and strip whitespace
- If file not found → raise `EmptyUserStory("user_story.txt not found")`
- If file empty after stripping → raise `EmptyUserStory("user_story.txt is empty")`
- Print `[STEP 2] User story loaded ... Done ✓`

STEP 3 — Fill prompt template at runtime NEW — Prompt Templates

- Define `PROMPT_TEMPLATE` string constant using `{user_story}` placeholder at top of file
- Define `build_prompt(user_story)` function that fills it using `PROMPT_TEMPLATE.format(user_story=user_story)`
- System prompt must instruct Gemini: act as senior QA engineer, return exactly 5 test cases, raw JSON only — no markdown, no explanation, follow exact 7-field structure
- Use `{{ }}` to escape literal braces inside the template — otherwise `.format()` throws `KeyError`
- Print `[STEP 3] Prompt template filled ... Done ✓`

STEP 4 — Call Gemini API with retry mechanism NEW — Retry

- Define `call_gemini_with_retry(prompt, max_retries=3, wait_seconds=2)`
- Use a `for` loop with `range(max_retries)` — NOT `while True`
- On success → return raw response text immediately
- On failure → print `Attempt {n} failed. Retrying in 2 seconds...` → `time.sleep(2)`
- If all 3 attempts fail → raise `APICallFailed("All 3 attempts failed")` → exit cleanly
- Set `temperature=0.2` and `max_output_tokens=1000` explicitly
- Print `[STEP 4] Calling Gemini API (max retries: 3) ...`

STEP 5 — Validate and parse JSON response with retry NEW — Retry on bad JSON

- Define `validate_and_parse(raw_text)` — run checks in this exact order:
- Strip markdown fences: `.replace("```json", "").replace("```", "").strip()`
- Parse with `json.loads()` — if fails → raise `InvalidAIResponse("Not valid JSON")`
- Validate result is a `list` → raise `InvalidAIResponse("Expected a list")`
- Validate exactly `5` items → raise `InvalidAIResponse(f"Expected 5, got {len(result)}")`
- Validate each item has all **7 required fields** → raise `InvalidAIResponse(f"Missing fields: {missing}")`
- In main pipeline, wrap STEP 4 + STEP 5 together in a **combined outer retry loop** — max 3 attempts, 2s wait — if `InvalidAIResponse` fires, retry full API call + parse cycle
- Print `[STEP 5] Response validated ... Done ✓ → 5 test cases generated`

I recommend replacing the manual validation step with Pydantic validation.

STEP 6 — Print formatted terminal report Phase 1 Week 1+2

- Use `enumerate(start=1)` to number each test case
- Print all 7 fields per test case — steps as a numbered sub-list
- After all test cases print priority summary: `Total: 5 | High: X | Medium: X | Low: X`

STEP 7 — Save to .json file Phase 1 Week 3

- Save to `output/generated_test_cases.json` using `json.dump()` with `indent=4`
- Print `[STEP 7] Saved to output/generated_test_cases.json ... Done ✓`

STEP 8 — POST each test case to JSONPlaceholder Phase 1 Week 3

- Loop using `enumerate(start=1)`
- Define `api_status = ""` **inside** the loop — never globally
- POST each to `https://jsonplaceholder.typicode.com/posts` with headers `{"Content-Type": "application/json"}`
- Wrap each call in `try/except requests.exceptions.RequestException`
- On `status_code == 201` → increment `success_count`
- On network failure → increment `failed_count` inside `except` → print error → `continue`
- Final: `Total Posted: 5 | Success: X | Failed: X`

Code Structure Skeleton

```
import json, os, time, requests
import google.generativeai as genai
from dotenv import load_dotenv

PROMPT_TEMPLATE = """..{user_story}..""" # defined at top of file

class MissingAPIKey(Exception): pass
class EmptyUserStory(Exception): pass
class InvalidAIResponse(Exception): pass
class APICallFailed(Exception): pass

def load_api_key(): ...
def read_user_story(): ...
def build_prompt(user_story): ... # fills PROMPT_TEMPLATE
def call_gemini_with_retry(prompt): ... # retry on API failure
def validate_and_parse(raw_text): ... # strict 7-field validation
def print_report(test_cases): ...
def save_to_json(test_cases): ...
def post_to_api(test_cases): ...

if __name__ == "__main__":
    api_key = load_api_key() # STEP 1
    story = read_user_story() # STEP 2
    prompt = build_prompt(story) # STEP 3

    for attempt in range(1, 4): # combined retry loop
        try:
            raw = call_gemini_with_retry(prompt) # STEP 4
            test_cases = validate_and_parse(raw) # STEP 5
            break
        except InvalidAIResponse as e:
            print(f"Validation failed attempt {attempt}: {e}")
            if attempt < 3:
                time.sleep(2)
            else:
                print("All 3 validation attempts failed. Exiting.")
                exit()

    print_report(test_cases) # STEP 6
    save_to_json(test_cases) # STEP 7
    post_to_api(test_cases) # STEP 8
```

AI_Test_Case_Generator/

- config/
 ├── __init__.py
 └── settings.py
- exceptions/
 ├── __init__.py
 └── custom_exception.py
- models/
 ├── __init__.py
 └── test_case_model.py
- prompts/
 ├── __init__.py
 ├── system_prompt.py
 └── prompt_builder.py
- services/
 ├── __init__.py
 ├── gemini_service.py
 ├── validation_service.py
 ├── report_service.py
 └── api_service.py
- utils/
 ├── __init__.py
 ├── file_handler.py
 └── json_handler.py

Expected JSON Structure (Instruct Gemini to return this)

```
[
  {
    "test_case_id": "TC001",
    "title": "string",
    "preconditions": "string",
    "steps": ["step1", "step2", "step3"],
    "expected_result": "string",
    "priority": "High | Medium | Low",
    "status": "Not Run"
  }
]
```

Expected Console Output

```
=====
AI-POWERED TEST CASE GENERATOR PIPELINE
=====

[STEP 1] API key loaded ... Done ✓
[STEP 2] User story loaded from user_story.txt ... Done ✓
[STEP 3] Prompt template filled ... Done ✓
[STEP 4] Calling Gemini API (max retries: 3) ...
        Attempt 1 succeeded ✓
[STEP 5] Response validated ... Done ✓ → 5 test cases generated
```

GENERATED TEST CASES REPORT

```
Test Case 1 : TC001
Title       : Login with valid credentials
Priority    : High
Precondition: User is registered and account is active
Steps      :
    1. Open browser
    2. Navigate to login page
    3. Enter valid email and password
    4. Click the Login button
Expected   : User is redirected to dashboard
Status     : Not Run
```

```
Test Case 2 : TC002
Title       : Login with invalid password
Priority    : High
Precondition: User is registered
Steps      :
    1. Open browser
    2. Navigate to login page
    3. Enter valid email, wrong password
    4. Click the Login button
Expected   : Error message is displayed
Status     : Not Run

... (repeat for TC003, TC004, TC005)
```

Total: 5 | High: 2 | Medium: 2 | Low: 1

[STEP 7] Saved to output/generated_test_cases.json ... Done ✓

[STEP 8] Posting to JSONPlaceholder API...

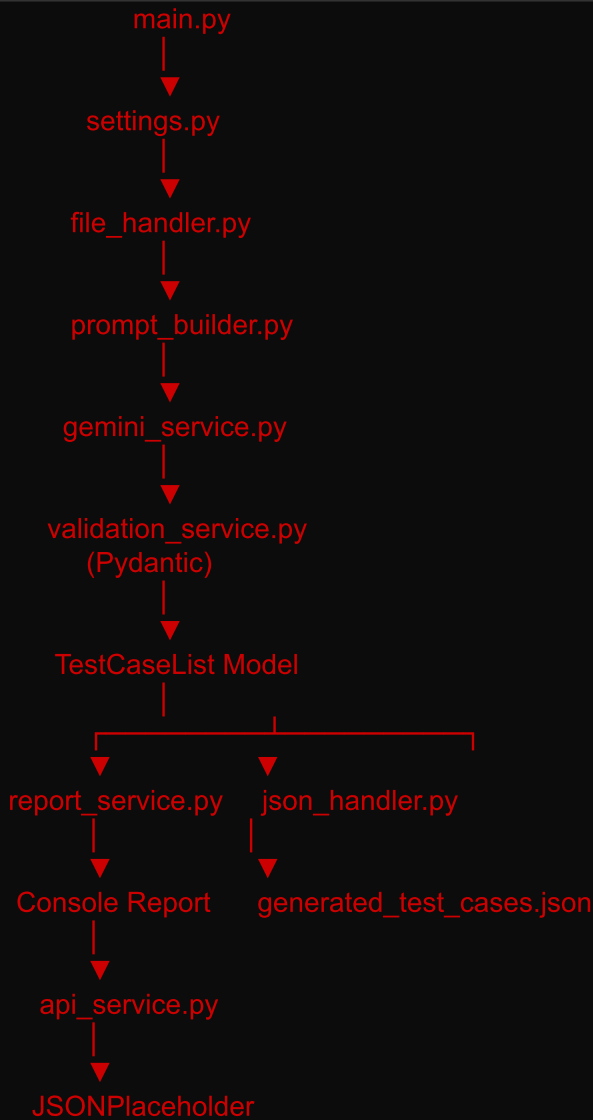
POSTING REPORT

```
Test Case 1 : Login with valid credentials
API Status  : Request Sent Successfully
Status Code : 201
Generated ID: 101

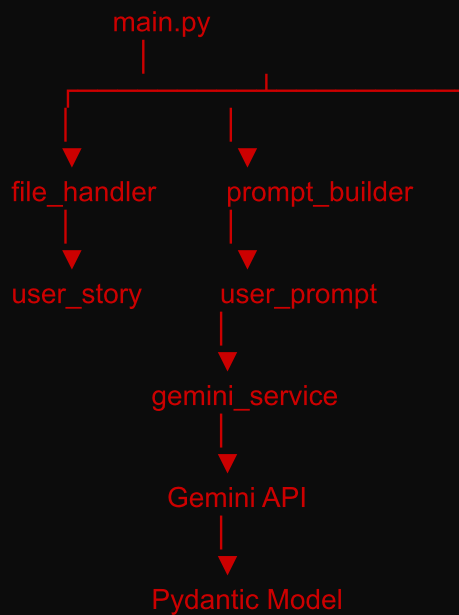
Test Case 2 : Login with invalid password
API Status  : Request Sent Successfully
Status Code : 201
Generated ID: 102

... (repeat for TC003, TC004, TC005)
```

Total Posted: 5 | Success: 5 | Failed: 0



The overall flow



What the console looks like when retry triggers (STEP 4 failure):

```
[STEP 4] Calling Gemini API (max retries: 3) ...
    Attempt 1 failed: Connection timeout. Retrying in 2 seconds...
    Attempt 2 failed: Connection timeout. Retrying in 2 seconds...
    Attempt 3 succeeded ✓
```

What the console looks like when JSON validation retry triggers (STEP 5 failure):

json_handler

JSON file

api_service

External API

```
[STEP 5] Parsing response ...
    Validation failed on attempt 1: Expected 5, got 4
    Retrying full API call + parse...
[STEP 4] Calling Gemini API (max retries: 3) ...
Attempt 1 succeeded ✓
[STEP 5] Response validated ... Done ✓ → 5 test cases generated
```

Acceptance Criteria Checklist

#	Criteria
1	API key loaded from <code>.env</code> — never hardcoded in script
2	<code>MissingAPIKey</code> raised if key is absent or empty
3	Two separate <code>EmptyUserStory</code> messages — file not found vs file empty
4	<code>PROMPT_TEMPLATE</code> defined as constant with <code>{user_story}</code> placeholder
5	<code>build_prompt()</code> fills template using <code>.format()</code> — literal braces escaped with <code>{{ }}</code>
6	<code>call_gemini_with_retry()</code> retries max 3 times with 2s wait using <code>for</code> loop — not <code>while True</code>
7	<code>APICallFailed</code> raised if all 3 API attempts fail
8	Markdown fences stripped before <code>json.loads()</code>
9	Validates: is list → has 5 items → each has all 7 fields — separate error message per check
10	Combined outer retry loop retries full API + parse cycle up to 3 times on <code>InvalidAIResponse</code>
11	Terminal report uses <code>enumerate()</code> — numbered steps + priority summary line
12	Output saved to <code>output/generated_test_cases.json</code> with <code>indent=4</code>
13	Each test case POSTed individually — <code>api_status</code> defined inside loop not globally
14	<code>failed_count</code> incremented inside <code>except</code> block of POST loop
15	All 8 functions defined — each does exactly one job

Install Before You Start

```
pip install google-generativeai python-dotenv requests
```

```
pip install google-genai pydantic python-dotenv requests
```

.env file:

```
GEMINI_API_KEY=your_actual_key_here
```

🔥 What Makes This Hard

- **4 custom exceptions** — not 3 like Phase 2 Week 2
- **Two nested retry mechanisms** — inner retry inside `call_gemini_with_retry()` for API failures, outer retry loop for combined API + JSON validation — must not conflict
- **7-field validation** — Gemini sometimes returns 6 fields or renames them — your validator must catch every case
- **Prompt template discipline** — `{{ }}` must escape literal braces or `.format()` throws `KeyError`
- **One function = one job** — 8 focused functions — first time your code is structured like a real production script

Notes before you start:

- Test each STEP in isolation — run STEP 1+2 first before touching the API
- Always strip markdown fences before `json.loads()` — Gemini often wraps output in ````json`
- This is a real API call that costs tokens — use a short user story during testing
- `pip install -r requirements.txt` before running